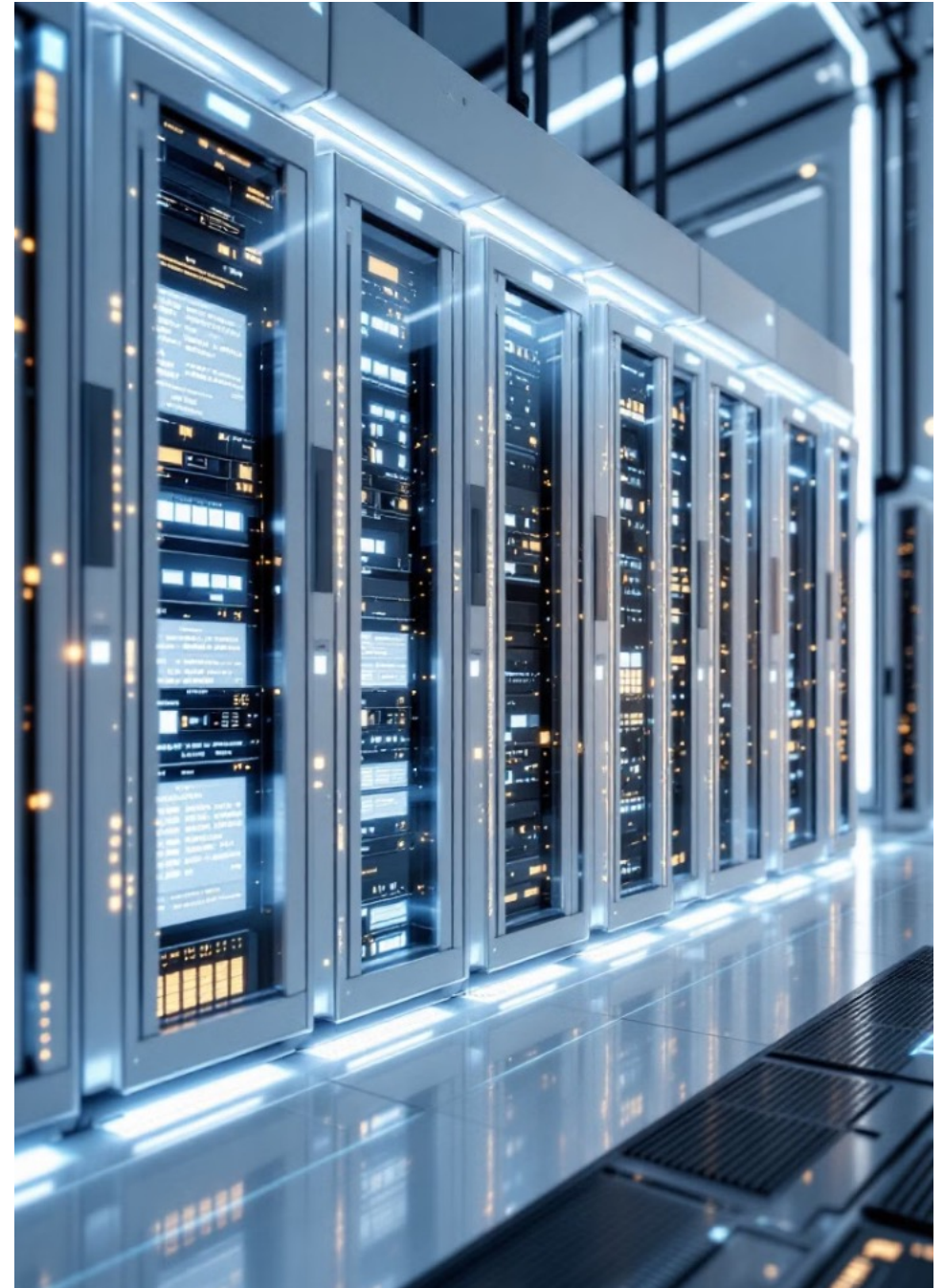


Caching in LLMs

1. Background.
2. Pipeline and opportunities.



1966: Eliza

“While ELIZA was capable of engaging in discourse, it could not converse with true understanding. However, **many early users were convinced of ELIZA's intelligence and understanding**, despite Weizenbaum's insistence to the contrary.”

Source: en.wikipedia.org/wiki/ELIZA (and references therein).

```
Welcome to

EEEEEE LL      IIII  ZZZZZZ  AAAAA
EE      LL      II    ZZ    AA   AA
EEEEEE LL      II    ZZZ   AAAAAA
EE      LL      II    ZZ    AA   AA
EEEEEE LLLLLL IIII  ZZZZZZ  AA   AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

Image source: en.wikipedia.org/wiki/ELIZA#/media/File:ELIZA_conversation.png

2005: Scigen automatic scientific paper generator.

A project using a rather rudimentary technology that aimed to "maximize amusement, rather than coherence" is still the cause of troubles today...

MIT News

ON CAMPUS AND AROUND THE WORLD

 [SUBSCRIBE](#)

How three MIT students fooled the world of scientific journals

A decade later, CSAIL alumni reflect on their paper generator and reveal a new fake-conference project.

 Watch Video

Adam Conner-Simons | CSAIL
April 14, 2015

news.mit.edu/2015/how-three-mit-students-fooled-scientific-journals-0414

nature

[Explore content](#) ▾ [About the journal](#) ▾ [Publish with us](#) ▾ [Subscribe](#)

[nature](#) > [news](#) > article

NEWS | **27 May 2021** Clarification [28 May 2021](#)

Hundreds of gibberish papers still lurk in the scientific literature

The nonsensical computer-generated articles, spotted years after the problem was first seen, could lead to a wave of retractions.

nature.com/articles/d41586-021-01436-7

2017: Google Revolutionized Text Generation



Google introduced the **Transformer**, which rapidly became the **state-of-the-art** approach to solve most NLP problems.

OpenAI's **G**enerative **P**re-trained **T**ransformer (DALL.E, 2021; ChatGPT, 2022), as the name suggests, reposes on Transformers.

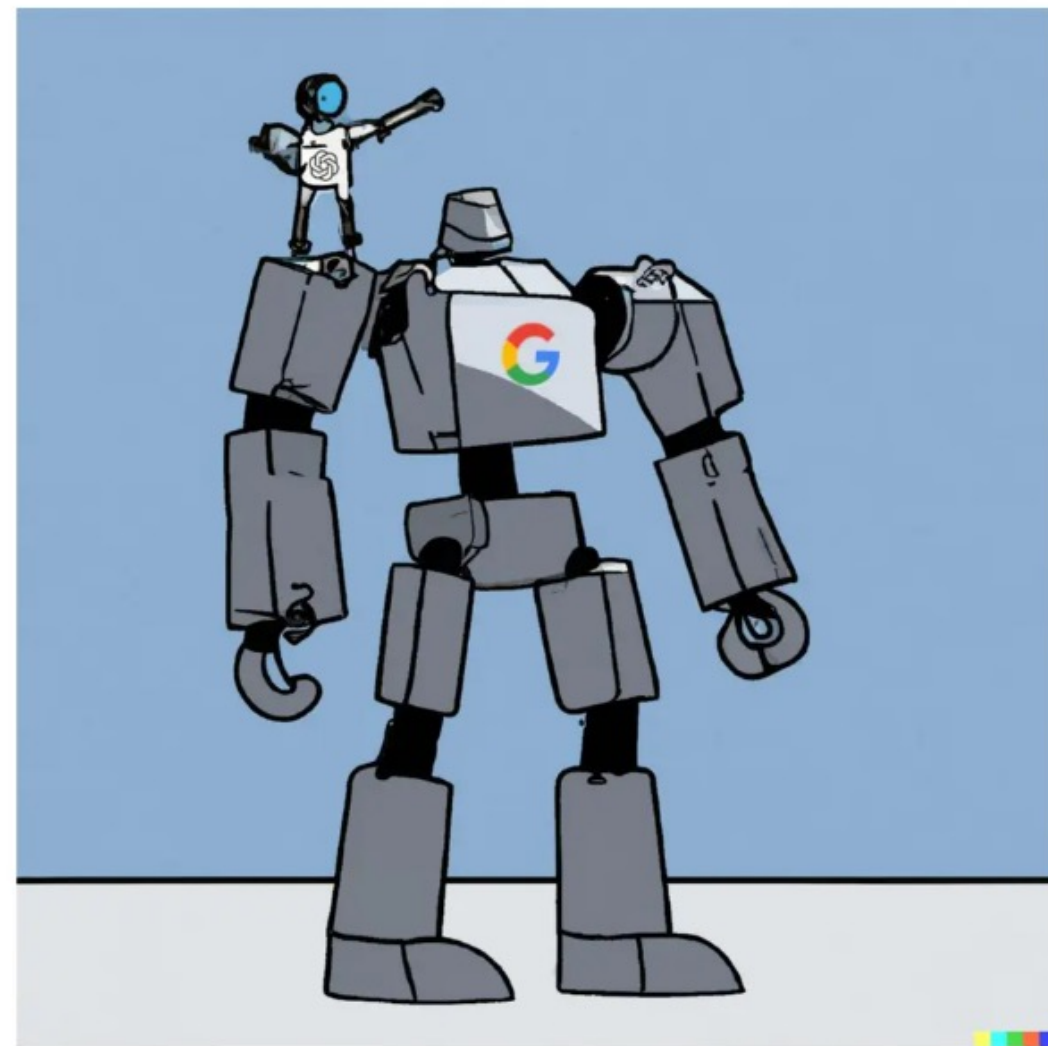
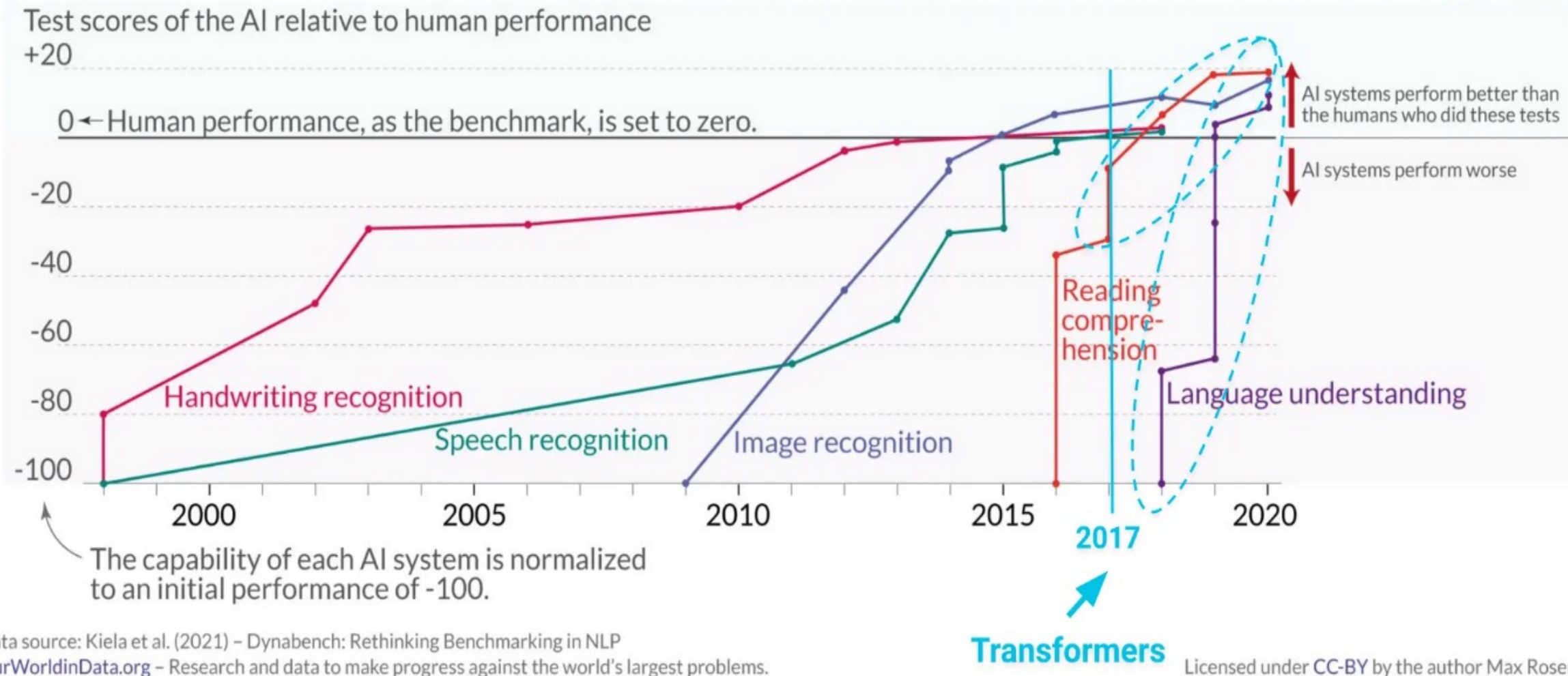


Image generated with DALL.E: "A small robot standing on the shoulder of a giant robot" (and slightly modified with The Gimp)

Language and image recognition capabilities of AI systems have improved rapidly



- Kiela et al. (2021), Dynabench: Rethinking Benchmarking in NLP: arxiv.org/abs/2104.14337
- Roser (2022), The brief history of artificial intelligence: The world has changed fast – what might be next?: ourworldindata.org/brief-history-of-ai

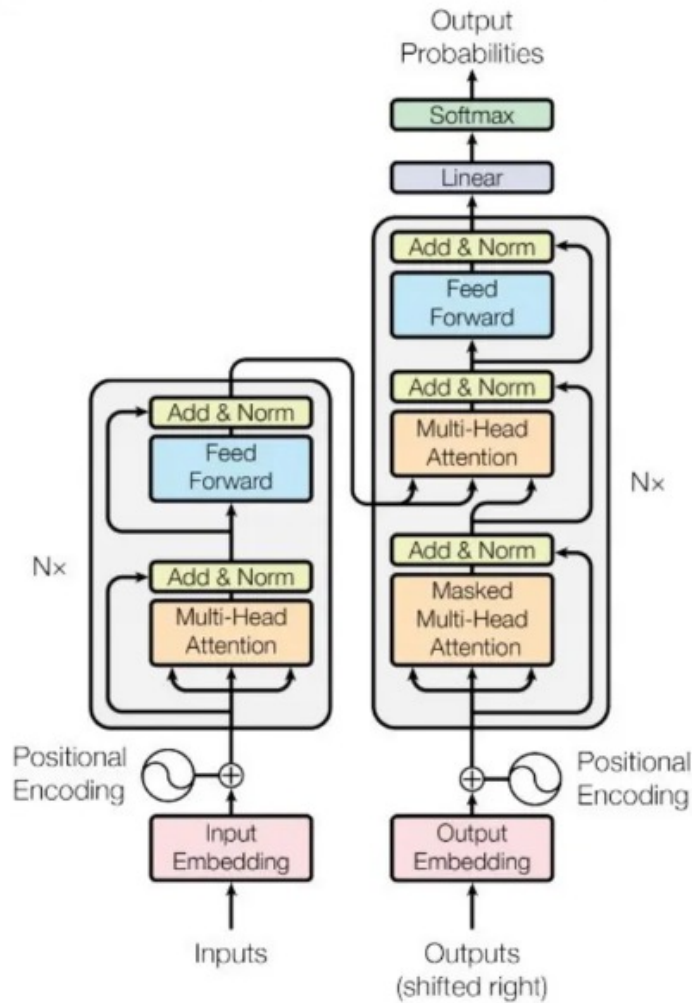
What Are Transformers?

Models that learn from a given dataset how to generate new data instances.

Generative (deep learning) **models** for understanding and generating text, images and many other types of data.

Transformers analyze chunks of data, called "tokens" and **learn to predict the next token** in a sequence, **based on previous and**, if available, **following tokens**.

The **auto-regressive** concept means that the output of the model, such as the prediction of a word in a sentence, is influenced by the previous words it has generated.



Source: Vaswani (2017), Attention Is All You Need
(doi.org/10.48550/arXiv.1706.03762)

Music—MusicLM (Google) and Jukebox (OpenAI) generate music from text.

Image—Imagen (Google) and DALL.E (OpenAI) generate novel images from text.

Text—OpenAI's GPT has become widely known, but other players have similar technology (including Google, Meta, Anthropic and others).

Others—Recommender (movies, books, flight destinations), drug discovery...

2022: ChatGPT

“**ChatGPT**, the popular chatbot from OpenAI, is estimated to have reached 100 million monthly active users in January, just two months after launch, making it the **fastest-growing consumer application in history**”

Reuters, Feb 1, 2023
<https://reut.rs/3yQNIGo>

ChatGPT Sprints to One Million Users

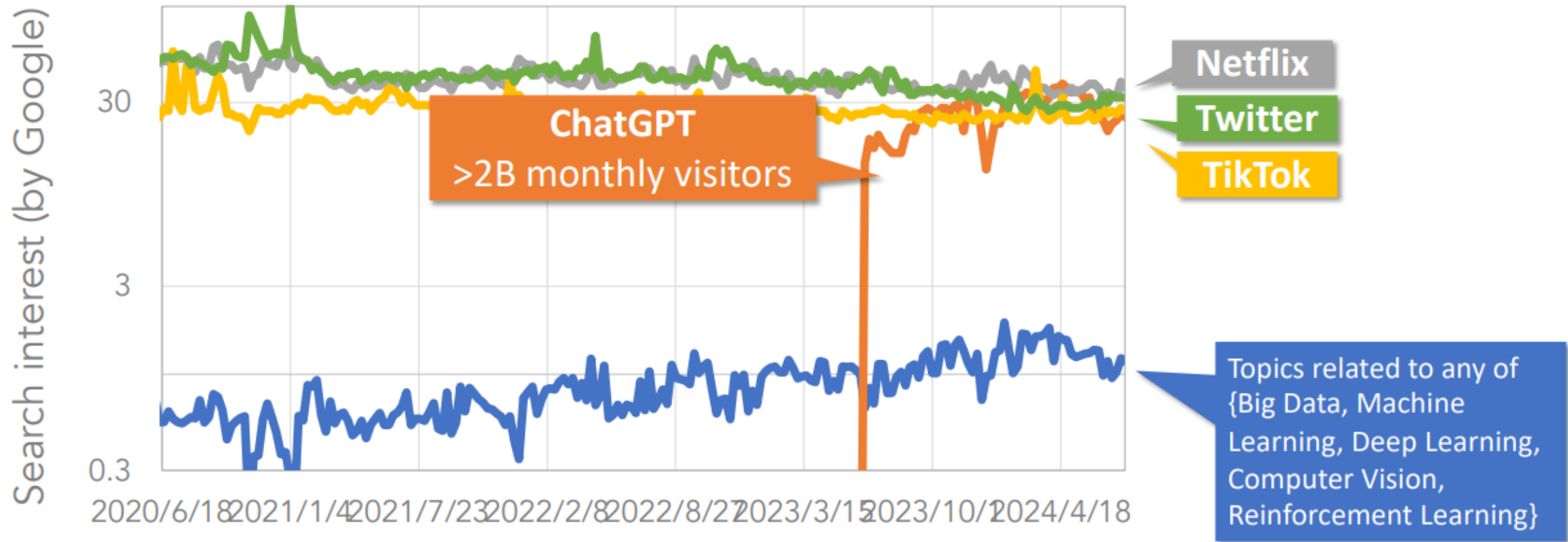
Time it took for selected online services to reach one million users



* one million backers ** one million nights booked *** one million downloads

Source: Company announcements via Business Insider/LinkedIn

LLMs are no longer just yet another ML app!

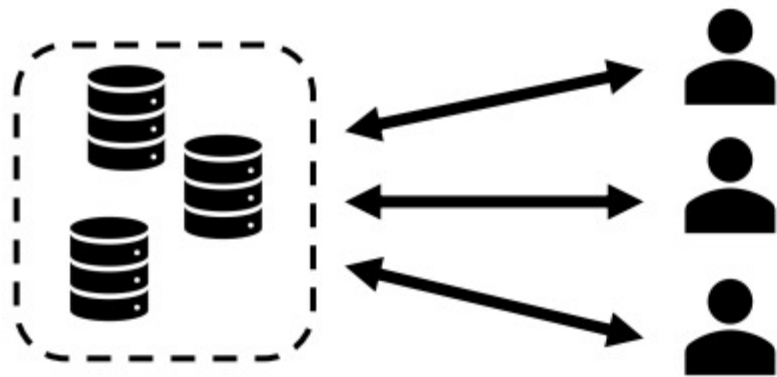


LLM applications are more like an Internet-scale service than other ML apps (CV, RL GAN, etc)!

How to build Internet-scale services?

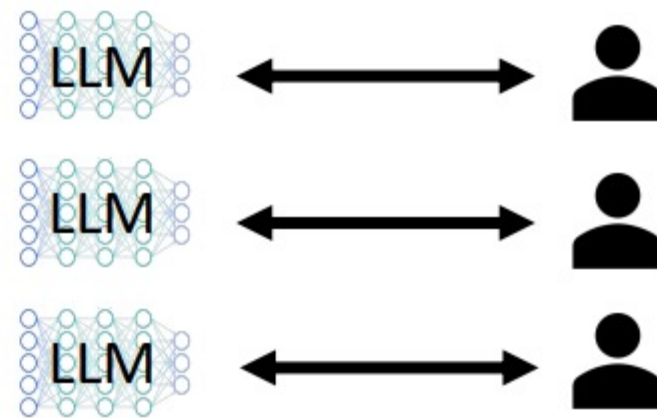
Internet-scale services

Leverage **cross-user/session** workload patterns emerging at the Internet scale



e.g., caching popular items

Yet, LLM systems research today
treat sessions or users **separately**



Lots of untapped opportunities in LLM systems!

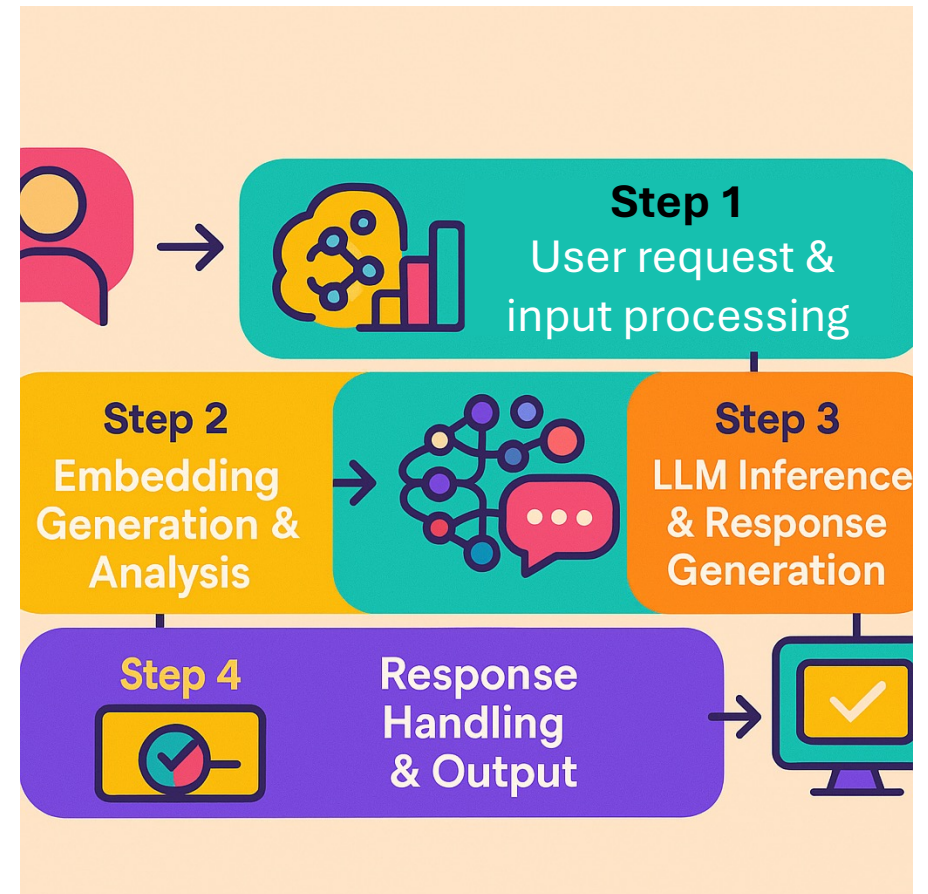
Lets get busy: LLM pipeline.

Step 1: User Request & Input Processing

Step 2: Embedding Generation & Semantic Analysis

Step 3: LLM Inference & Response Generation

Step 4: Response Handling & Output



Step 1: User Request & Input Processing

Workflow: A user initiates a request or query to a Large Language Model (LLM). This could be a single-round question-answering task or a multi-turn conversation.

Input Processing: The LLM request undergoes initial processing. This may involve formatting the input into a specific protocol.

Pre-processing can also occur, such as removing prompt information, compressing the input, or retaining only the most recent parts of a long text or conversation.

- **Caching Opportunity: Prompt Caching** can be implemented at this stage. If a new prompt is identical or semantically similar to a previous prompt for which a response is already available, the cached response can be directly reused, avoiding the need to call the LLM. Techniques like **embedding similarity** can be used to determine if prompts can be answered by the same response.

Step 2: Embedding Generation & Semantic Analysis

Workflow: If a cache hit does not occur based on the initial input, the LLM (or a component within the caching system) typically generates an embedding vector for the input prompt. This vector represents the semantic meaning of the query.

Semantic Analysis: The embedding vector of the current query is compared against the embeddings of previously processed and cached queries, often using cosine similarity. This step aims to identify semantically similar queries that might have the same or similar answers.

Caching Opportunities:

Caching Embedding Vectors: The generated embedding vectors themselves can be stored and indexed in a vector database (e.g., VecDB) for efficient similarity search.

Semantic Caching: By comparing embeddings, the system can identify if a semantically similar query exists in the cache, even if the new query is not an exact match to a previous one. If a similar query with a satisfactory response is found, that response can be retrieved. This requires effective methods to predict whether semantically similar prompts can be answered by the same response.

Step 3: LLM Inference & Response Generation

Workflow: If no suitable cached response is found based on semantic similarity, the LLM proceeds with the computationally intensive process of inference. This involves processing the input through its neural network layers to generate a contextually relevant response.

Resource Intensive: LLM inference can be time-consuming and requires significant computational resources, including GPU usage and processing of a substantial number of tokens.

Caching Opportunities:

Caching LLM Responses: The generated response from the LLM can be stored in the cache, associated with the original query (or its embedding). This ensures that if the exact same or a semantically similar query is received in the future, the pre-computed response can be served directly, saving computation time and costs.

-

Neural Cache Model: A "Neural Cache Model" proposes storing recent hidden activations of the neural network during inference. These stored activations can be used as representations of the context and, through a dot-product with the current hidden activations, can inform the prediction of the next word. This approach requires no training and can be added to any pre-trained neural network.

Step 4: Response handling and Output

Workflow: Once the LLM generates a response (or a cached response is retrieved), it is handled and prepared for output to the user. This might involve formatting the response according to the application's requirements.

Contextual Considerations: For contextual or follow-up queries, the system might need to consider the history of the conversation to provide accurate responses.

Caching Opportunities:

Contextual Caching: Caching should ideally account for the context of the conversation. Even if a new query is semantically similar to a past one, the response might differ based on the preceding conversation. Some caching systems record the contextual chain of queries to avoid false hits for contextually different but semantically similar queries.

Multimodal Response Caching: As LLMs evolve to handle multimodal inputs and outputs (e.g., images, videos), caching mechanisms can be extended to store and reuse these more complex responses, which can be particularly beneficial due to their potentially long generation times.

Summary: Caching Opportunities.

Step 1: User Request & Input Processing:

1. Prompt caching.

Step 2: Embedding Generation & Semantic Analysis:

2. Caching embedding vectors.
3. Semantic caching.

Step 3: LLM Inference & Response Generation

4. Caching LLM responses.
5. Neural Cache Model.

Step 4: Response Handling & Output

6. Contextual Caching
7. Multimodal Response Caching